3D printer improvements

From the community for the community, mostly via heavily customized tools

Some links about me. Many of my 3D designs are free. I also post on Google+ and in another blog, oz4.us

Sticky posts

survival kit, openscad tutorial, material for artists, what cannot be 3D printed, oz4.us...

Tweets by tridimake

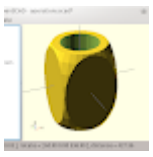
Popular posts

Replacing LinuxCNC with a Smoothieboard for CNC milling. Not as easy as for 3D printing!

I was running LinuxCNC for the last years and I wanted a smaller setup, with room for improvement. Since I had a Smoothieboard around, whi...

How to use Openscad (1), tricks and tips to design a parametric 3D object

Part 1/5: Introduction to constructive solid geometry with Openscad Some technical and non-technical people keep asking me how I create...

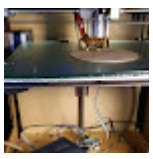


How to use Openscad (2): variables and modules for parametric designs

Part 2/5: Variable and parametric design The previous part addressed the basics of Openscad. It relied mostly on "immediate values...

How to use Openscad (4): children and advanced topics

Part 4/5: children, factorized placement and chained hulls Yes, it was made with Openscad and it is parametric! (extreme collaborativ...



Cooling 3D prints with an aquarium pump

3D printing cooled with an aquarium pump! It works well and it cannot be quieter. T he pump is bulky, but the overall is a very compact...

review (32) general (26) issue (16) mechanical (16) replacement (14) software (13) addon (12) bowden tube (10) business (10) consumables (10) heat (10) feeder (8) hot end (8) weird (7) motor (6) off-topic (6) trick (6) electric (4) bed (3) cnc (2)

Sunday, November 23, 2014

How to use Openscad (4): children and advanced topics

Part 4/5: children, factorized placement and chained hulls

Previously in this tutorial for the Openscad CAD software, we talked only about modules that behaved as *shapes*.

Yes, it was made with Openscad and it is parametric!
(extreme collaborative work, picture by N.Goodger)

A powerful and often ignored feature of Openscad is that modules can also behave as if they were *operators*, exactly like the `translate()` or `color()` operators. They do not create shapes on their own, but they modify the subsequent commands.

In Openscad, it is possible through the use of **children**. But first, let us create and discuss a bit about a common-case example.

This article is part of a longer serie:

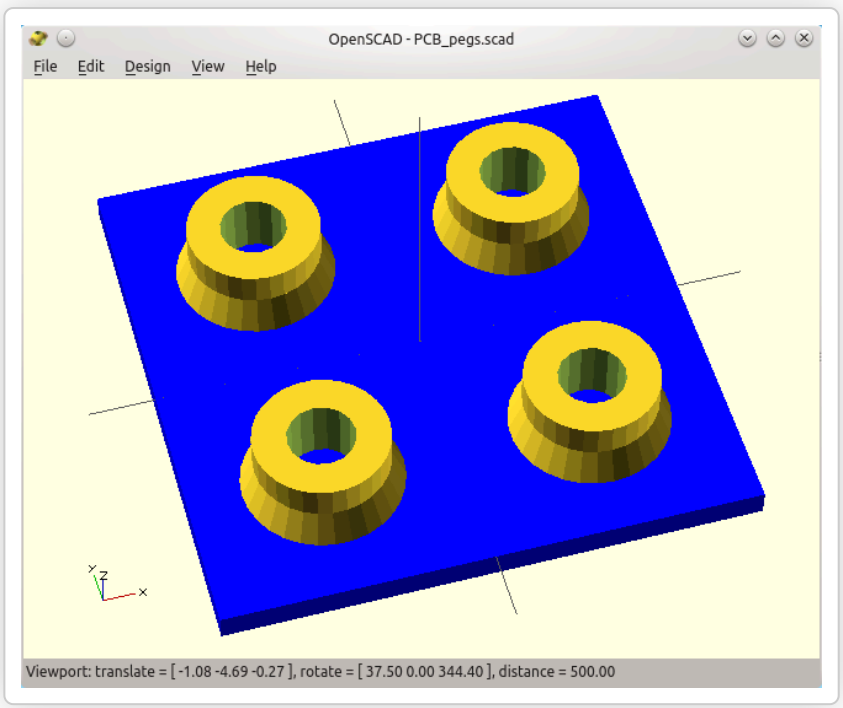
1. Introduction to constructive solid geometry with OpenSCAD
2. Variables and modules for parametric designs
3. Iteration, extrusion and useful parametrized CSG techniques
4. **Children, factorized placement and chained hulls**

How can "negative" shapes extend outside of their module?

This is a common design problem with the constructive solid geometry paradigm.

Let us consider a support for an electronic board. Each of the "pegs" is made of a cylinder, a bevel at the base, and a hollow cylinder for the screw. We conveniently put all these in a "peg" module.

But we want the screws to go all through the support, which itself it added outside of the module...



Failed! The peg holes do not go through the blue plate...

Forget about the dimensions, we do not care here.

As we have seen, it is a good practice to factorize the peg shapes into a module, and then to iterate four times on it, like this:

```
screw_dist=40; // PCB hole spacing (square)
height=10; // peg height
peg_d=20; // peg diameter
screw_d=10; // hole diameter
plate_th=1; // plate thickness
tol=0.05; // used for CSG subtraction/addition

module one_peg()
{
  difference()
  {
    union()
    {
      // main cylindrical body
      cylinder(r=peg_d/2,h=height);
      // with a tronconic base (bevel)
      cylinder(r1=peg_d/2+2,r2=peg_d/2,h=5);
    }
    translate([0,0,-tol]) // screw hole
      cylinder(r=screw_d/2,h=height+2*tol); // through hole
  }
}

// Helper module: centered cube on (X,Y) but not on Z, like cylinder
module centered_cube(size)
{
  translate([-size[0]/2, -size[1]/2, 0])
    cube(size);
}

// Now call and build the four pegs
for(r=[0:90:359])
  rotate([0,0,r])
    translate([screw_dist/2,screw_dist/2,0])
      one_peg();

color([0,0,1])
  translate([0,0,-plate_th])
    centered_cube([
      screw_dist+peg_d*2,
      screw_dist+peg_d*2,
      plate_th+tol]); // slightly higher to avoid gaps
```

But when we add the blue bottom plate, flush with the pegs, the holes are no more open!

So how do we fix this?

Making holes longer will not help, because they apply only to the shapes defined in the `one_peg` module (i.e. only the cylinders and before the plate is added).

A way out could be to take the negative (hollow) cylinder out of the module, and remove them to the union of the pegs

and the plate. But it breaks the very idea of a module by sharing "internal" information with the outside (it crosses the module encapsulation). It also makes the source code not very readable and less parametric.

Or the module itself could be written with a "positive" and "negative" version like this. The changes are highlighted.

```
screw_dist=40; // PCB hole spacing (square)
height=10; // peg height
peg_d=20; // peg diameter
screw_d=10; // hole diameter
plate_th=1; // plate thickness
tol=0.05; // used for CSG subtraction/addition

module one_peg(positive_shape=true)
{
  if(positive_shape)
  {
    // main cylindrical body
    cylinder(r=peg_d/2,h=height);
    // with a tronconic base (bevel)
    cylinder(r1=peg_d/2+2,r2=peg_d/2,h=5);
  }
  else // "negative" shape (carve the inner peg cylinder)
  {
    translate([0,0,-tol -plate_th]) // screw hole
      cylinder(r=screw_d/2,h=height+2*tol + plate_th); // hole
  }
}

// Helper module: centered cube on (X,Y) but not on Z, like cylinder
module centered_cube(size)
{
  translate([-size[0]/2, -size[1]/2, 0])
    cube(size);
}

difference()
{
  union()
  {
    // Now call and build the four pegs positive shapes (not carved)
    for(r=[0:90:359])
      rotate([0,0,r])
 translate([screw_dist/2,screw_dist/2,0])
   one_peg(positive_shape=true);

    color([0,0,1])
      translate([0,0,-plate_th])
 centered_cube([
   screw_dist+peg_d*2,
   screw_dist+peg_d*2,
   plate_th+tol]); // higher to avoid gaps with the pegs!
  }

  // And finally carve the pegs (all through the plate itself)
  for(r=[0:90:359]) // repeat the positioning
    rotate([0,0,r])
      translate([screw_dist/2,screw_dist/2,0])
 one_peg(positive_shape=false);
}
```

Now there is something annoying: positioning the "positive" peg shapes and the "negative" peg shapes rely on fully duplicated code. This is bad.

Indeed, any change in the positioning would need to be duplicated. Bug fixes also! Duplicating code adds uninteresting codelines and it is prone to bugs, because a bug fix may be forgotten as the programmer does not know where and how many times the code was duplicated. It soon becomes very hard to maintain and upgrade such a design!

This is why I often use a "placement" module for the location of the pegs.

Referring to the children of a module: how it works

This is a somehow tricky but very powerful Openscad feature.
It is not usually found in other languages, which may be the reason why it is too often overlooked by programmers who use openscad.

Now, it is extremely powerful as it makes the source code much cleaner, and it allows many optimizations.

In practice, when you are in a module, Openscad creates two special variables for you, that refer to the shapes that are in the *scope* of the module when it gets used.

`children` - an vector that holds reference to each of shapes that are impacted by the module

`$children` holds the number of shapes that *follows* the module use

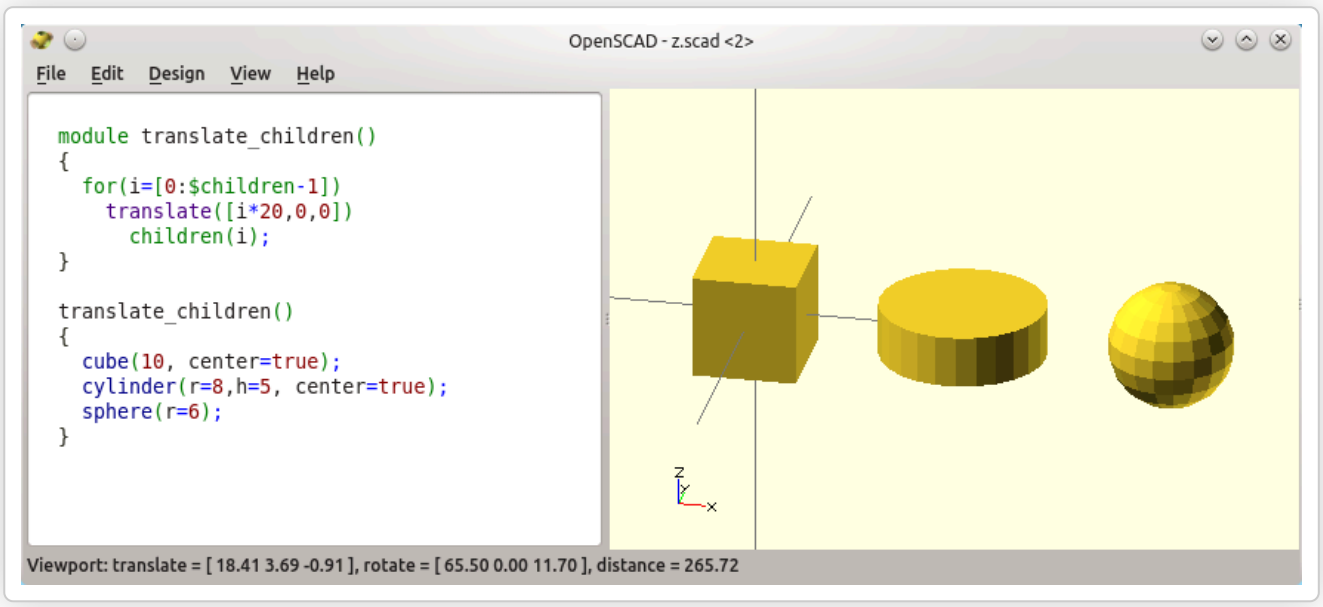
To get an indivudual value of a vector in Openscad, you can use this syntax:

`children(i)`

Now, the first children of the module will start at index zero (not one!). So the last children really is `children($children-1)`. Accessing the vector at index `$children` is an error, just like trying to use a negative number. This is the norm in most of the programming languages when dealing with arrays: the first item is at index zero.

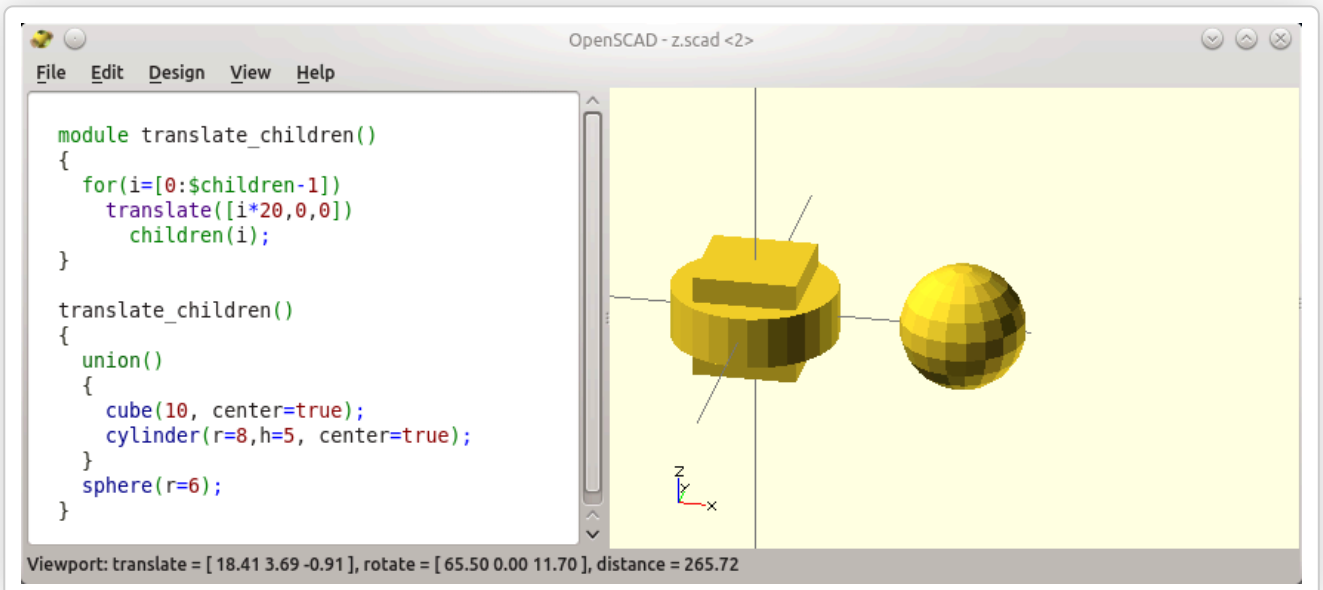
So what is a child now? It is any primitive or composed shape that appear right "under" the module call itself.

Check the example below, where `translate_children()` iterates on its children, so as to translate each of them on the X axis by a value which is proportional to their order of appearance, before "calling them":



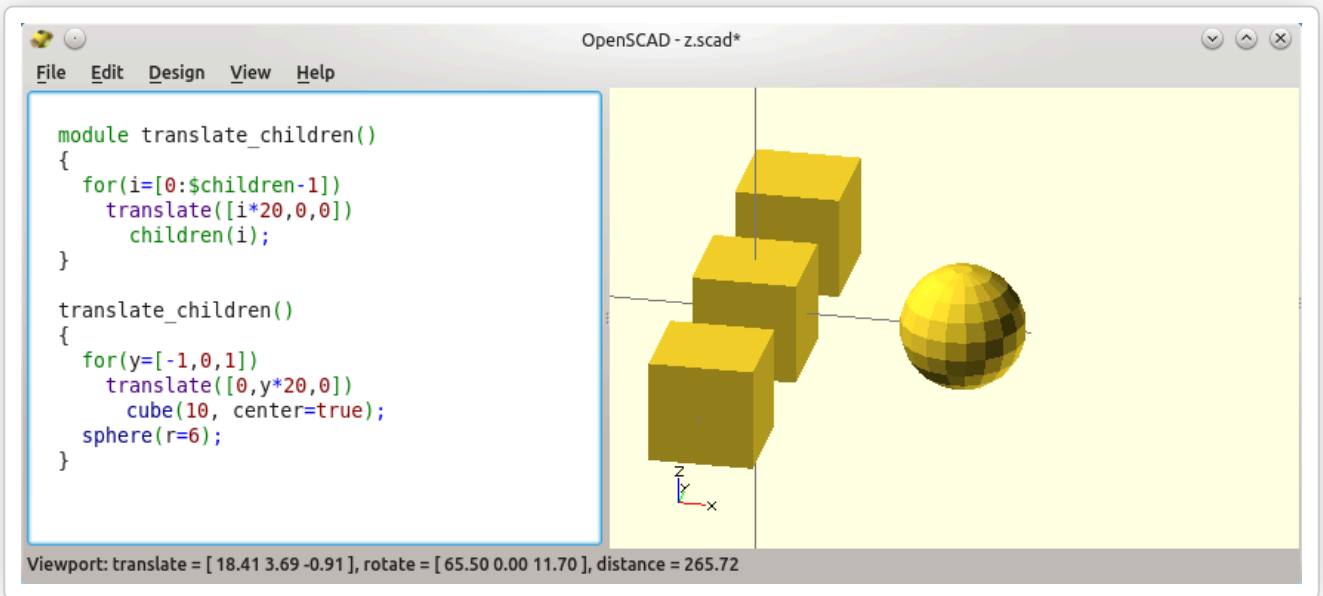
Efficiently translating the "children" of a module proportionally to their order of appearance!

Now, a children really is a shape that is "just below" the module. In the following, the cube and the cylinder are seen from within the module as a single child object (which they really are!)



Children are really only at the top level withing a module!

For the same reason, the iteration below is also one single "unioned" object for the module. It does not see the individual sub-shapes!



Hence, the $children feature does not "see" the individual cubes either (they were merged beforehand)!

The last case occurs because `for` does an implicit union on the items. This problem is highlighted by the weird but required `intersection_for()` in the official documentation, that will be of no help here though...

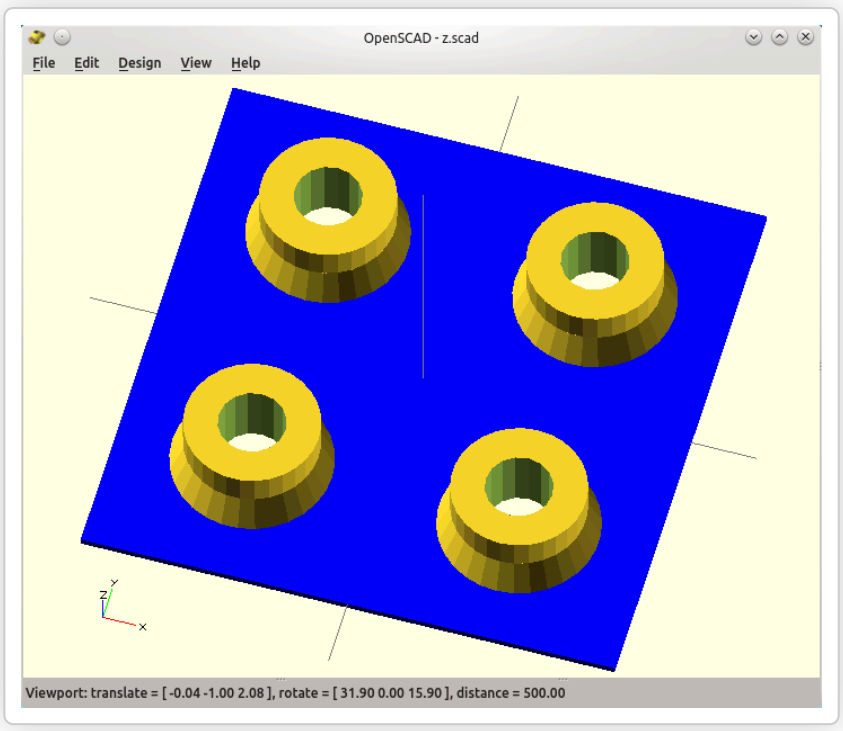
A module as a positioning tool: using the *children* array

Back to our PCB holder... Thanks to Openscad's `$children` feature, we now can *factorize both the duplication and the placement* of each of the four individual pegs. Here again the changes are highlighted when compared to the previous source code.

We do it this way:

```
screw_dist=40; // PCB hole spacing (square)
height=10; // peg height
peg_d=20; // peg diameter
screw_d=10; // hole diameter
tol=0.05; // used for CSG subtraction/addition
plate_th= 1;

module peg_positions()
{
  for(r=[0:90:359]) // it will make 4 versions of the sub-shapes
    rotate([0,0,r])
      translate([screw_dist/2,screw_dist/2,0])
      {
        for(i=[0:$children-1])
          children(i); // do each of the shapes that come after us
      }
}

module one_peg(positive_shape=true)
{
  if(positive_shape)
  {
    // main cylindrical body
    cylinder(r=peg_d/2,h=height);
    // with a tronconic base (bevel)
    cylinder(r1=peg_d/2+2,r2=peg_d/2,h=5);
  }
  else
  {
    translate([0,0,-tol -plate_th]) // screw hole
      cylinder(r=screw_d/2,h=height+2*tol + plate_th); // hole
  }
}

// Helper module: centered cube on (X,Y) but not on Z, like cylinder
module centered_cube(size)
{
  translate([-size[0]/2, -size[1]/2, 0])
    cube(size);
}

difference()
{
  union()
  {
    // Now call and build the four pegs
    peg_positions()
      one_peg(positive_shape=true);

    color([0,0,1])
      translate([0,0,-plate_th])
        centered_cube([
          screw_dist+peg_d*2,
          screw_dist+peg_d*2,
          plate_th+tol]); // higher to avoid gaps with the pegs!
  }

  // And finally carve the pegs (all through the plate itself)
  peg_positions()
    one_peg(positive_shape=false);
}
```

What happens here? The module `peg_position()` does not create any shape on its own. Instead it does all the work to duplicate and position each of the pegs, and only then it refers to its own children to do something. We have a unique children here each time, a call to the `one_peg()` module.



Here we are. Through-holes for the PCB pegs.

To conclude this part, we can merge everything in a single compact module, with an "action" argument. I generally do it this way:

```
screw_dist=40; // PCB hole spacing (square)
height=10; // peg height
peg_d=20; // peg diameter
screw_d=10; // hole diameter
tol=0.05; // used for CSG subtraction/addition
plate_th= 1;

module peg(action)
{
  if(action=="position")
  {
    for(r=[0:90:359])
      rotate([0,0,r])
        translate([screw_dist/2,screw_dist/2,0])
        {
          for(i=[0:$children-1])
            children(i); // do the shapes that come after us!
        }
  }
  else if(action=="add")
  {
    // main cylindrical body
    cylinder(r=peg_d/2,h=height);
    // with a tronconic base (bevel)
    cylinder(r1=peg_d/2+2,r2=peg_d/2,h=5);
  }
  else // "remove"
  {
    translate([0,0,-tol -plate_th]) // screw hole
      cylinder(r=screw_d/2,h=height+2*tol+plate_th); // through hole
  }
}

// Helper module: centered cube on (X,Y) but not on Z, like cylinder
module centered_cube(size)
{
  translate([-size[0]/2, -size[1]/2, 0])
    cube(size);
}

difference()
{
  union()
  {
    // Now call and build the four pegs
    peg("position")
      peg("add");

    color([0,0,1])
      translate([0,0,-plate_th])
        centered_cube([
          screw_dist+peg_d*2,
          screw_dist+peg_d*2,
          plate_th+tol]); // higher to avoid gaps with the pegs!
```

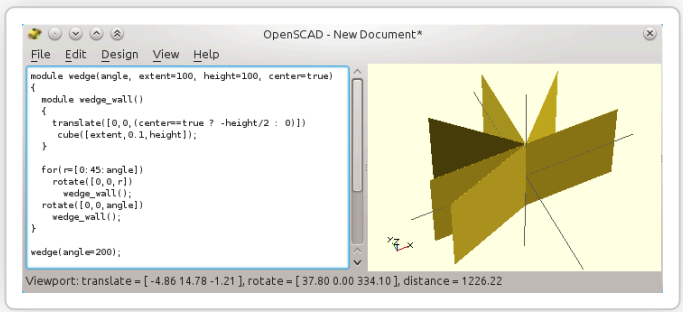
```
  }

  // And finally carve the pegs (all through the plate itself)
  peg("position")
    peg("remove");
}
```

The result is a code which is much more readable, and more useful. Still one has to split the added shape and the subtracted shape, and it must be taken into account. The "remove" part must be positioned correctly in the CSG tree, after the plate is merged with the positive peg shapes.

Now, whenever other shapes must be positioned relatively to the pegs, we can use the convenient `peg("position")` module call, and group the shapes in a union that will be "preprocessed" by the module.

Another very useful use of children : "Serial hull", aka "chained hull"

Remember the wedge in our previous post? It was built out of "support walls" to intersect a torus and leave only a variable sector. We had to revert to sequence of pairs of convex hulls in order to create the appropriate (concave) final shape.



The support structures we used to build a wedge shape that we used to intersect a torus made from a fully circular `rotate_extrude()`.

The deal here would be to create a module that does the union of a sequence of hulls in a list of children. Let us call this special module "serial_hull( )".

Calling it this way:

```
serial_hull()
{
  thin_wall1();
  thin_wall2();
  ...
  thin_wall7();
}
```

...would create a convex hull of the shapes 1 and 2, then another hull with the shapes 2 and 3, and so. The resulting convex hulls would then be merged (with the default union operation).

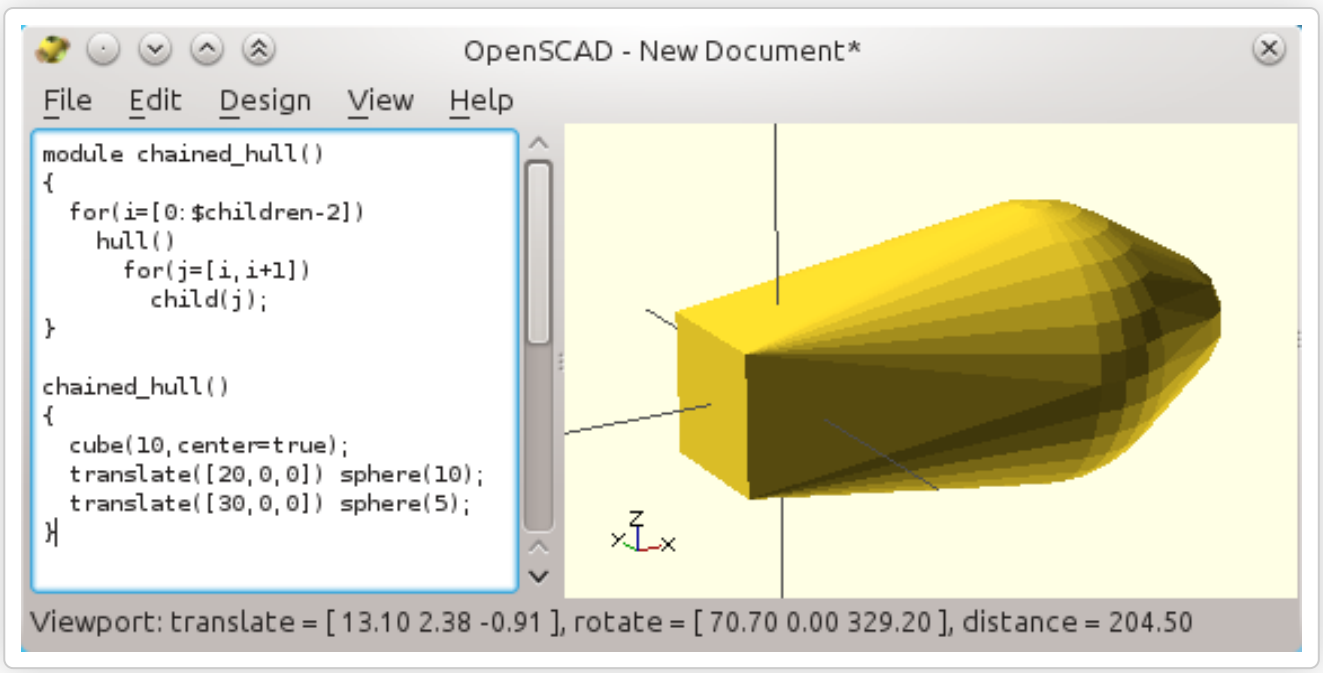
See how the union of many hulls is definitely *not* like the hull of all the shapes at once?

Using loops, it becomes straightforward to implement:

```
module serial_hull()
{
  // let i be the index of each of our children, but the last one
  for(i=[0:$children-2])
    hull() // we then create a hull between child i and i+1
    {
      children(i); // use child() in older versions of Openscad!
      children(i+1); // this shape is i in the next iteration!
    }
}
```

Note that we can now join any kind of shapes:

```
serial_hull()
{
  cube(10,center=true);
  translate([20,0,0]) sphere(10);
  translate([30,0,0]) sphere(5);
}
```



Linking a cube, a large sphere, and a smaller one with a union of consecutive convex hulls, the pretty way!

Note: if you are using an old version of Openscad you will have to use "child(index)" in place of "children(index)" (they just changed the name of the variable for more consistency).
Reciprocally, you can use "child" is a newer version, but you will get a warning stating "DEPRECATED: child() will be removed in future releases. Use children() instead.".

So will it help our parametric wedge, to build a partial "rotate extrude"?

Now let us try and go back to our partial torus... It will fail if we try naively...

Remember, we have seen the issue in the introduction to `$children` above: the module sadly considers the `for` loop on the `wedge_wall()` as a single child, and not as separate shapes.

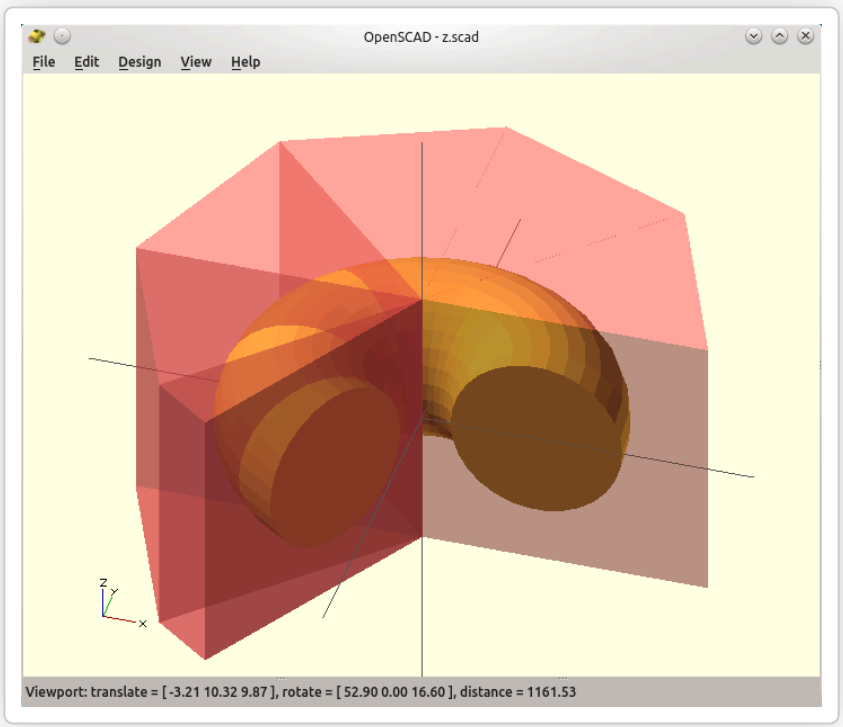
```
module chained_hull()
{
  for(i=[0:$children-2])
   hull()
   {
     children(i);
     children(i+1);
   }
}

module wedge(angle, extent=100, height=100, center=true)
{
  module wedge_wall()
  {
    translate([0,0,(center==true ? -height/2 : 0)])
      cube([extent,0.1,height]);
  }

  chained_hull() // will this create consecutive hulls? NO!!
  {
    for(r=[0:45:angle-1]) // implicit union of all the items!!
      rotate([0,0,r])
        wedge_wall();
    rotate([0,0,angle]) // this is only the second child!
      wedge_wall();
  }
}

wedge(angle=270);
```

In fact, and I would like to be corrected, I do not know how we could use `$children` and "exploded" `for` loops at the same time.

So far, the most compact version I could make is to build the angular sector this way, where some thin walls may be stacked on each other at angle "angle" (no harm with respect to CSG manifold policies).

```
module chained_hull()
{
  for(i=[0:$children-2])
    hull() // here we do it another way (without a secondary loop)
    {
      children(i);
      children(i+1);
```

```
    }
}

module wedge(angle, extent=100, height=100, center=true)
{
  module wegde_if(sub)
  {
    // Revert to angle zero in case the provided one "sub"
    // exceeds the wedge maximum angular size "angle":
    rotate([0,0,( angle>sub ? sub : angle)])
      translate([0,0,(center==true ? -height/2 : 0)])
        cube([extent,0.1,height]);
  }

  chained_hull() // explicit children makes it work...
  {
    // Create the appropriate thin walls each 45° around Z
    // They cannot be made with a "for" loop b/c of the implicit
    // union it does, which would fail with the chained_hull()
    wegde_if(0);
    wegde_if(45);
    wegde_if(45*1);
    wegde_if(45*2); // wedge_if() "flattens" sections that are
    wegde_if(45*3); // past the user-provided "angle", so only
    wegde_if(45*4); // the right number of sections will be
    wegde_if(45*5); // generated!
    wegde_if(45*6);
    wegde_if(45*7);
    wegde_if(45*8); // this last one is "chopped" to "angle"
  }
}

intersection()
{
  rotate_extrude(convexity=10)
    translate([40,0,0])
      circle(r=30);
#  wedge(angle=240); // "hash sign" to see it as below!
}
```



As can be seen, we eventually designed our partial but generic torus!

One last attempt to get rid of the manual repetitions: if/then readability issues

As usual there are numerous ways to achieve the same goal. We can use only convex wedges and decide either to `intersect` or to `subtract` it from the torus. This way we also can rely on for-loops to build the wedge and stay compatible with the `chained_hull()`...

But `if/else` constructs are usually considered bad practice because they disrupt the CSG tree.
Also, because of restrictions in the CSG algorithms, they cannot apply only to the *choice* of the operation (`intersection` versus `difference`), so as to keep a common "content".

For example, the following is illegal. It will not compile:

```
if(condition)
  intersection()
else
```

```
  difference()
{
  cube([100,100,100],center=true);
  sphere(r=55);
}
```

This is why using if/then does generally not help to keep the code factorized. It should be written like this, repeating the content twice :(

```
if(condition)
  intersection()
  {
    cube([100,100,100],center=true);
    sphere(r=55);
  }
else
  difference()
  {
    cube([100,100,100],center=true);
    sphere(r=55);
  }
```

This is why I tend to favor the former use of valued conditionals like `(test ? then_value : else_value)` where only *values* are changed and not the *operations*. There is a second reason in my opinion: it tends to be faster when rendering the design (probably because cache and optimization are more efficient on a static CSG tree).

Anyhow, here is the variant with only convex wedges, which is nonetheless more parametric (there are no more "hand placed" thin walls at each 45°).
By the way, we made the `wedge_this` now a modifier on its own:

```
module chained_hull()
{
  for(i=[0:$children-2])
    hull()
    {
      children(i);
      children(i+1);
    }
}

module wedge_this(angle, extent=100, height=100, center=true)
{
  module convex_wedge(a) // compact but works ONLY for a<=180°
  {
    translate([0,0,(center==true ? -height/2 : 0)])
      chained_hull()
      {
 for(r=[0:45:a-1]) rotate([0,0,r]) cube([extent,0.1,height]);
 rotate([0,0,a]) cube([extent,0.1,height]);
      }
  }

  if(angle<=180) // then intersect the wedge with the children
  {
    intersection()
    {
      for(i=[0:$children-1]) children(i);
      convex_wedge(angle);
    }
  }
  else // else subtract the reciprocal wedge from the children
  {
    difference()
    {
      for(i=[0:$children-1]) children(i); // union of the children
      scale([1,-1,1]) convex_wedge(360-angle); // reciprocal wedge
    }
  }
}

// Usage example:
wedge_this(angle=265)
  rotate_extrude(convexity=10)
    translate([40,0,0])
      circle(r=30);
```

I am not sure the if/then makes it more readable, but you just got more food to crunch! :)

About these tutorials and a (temporary?) conclusion...

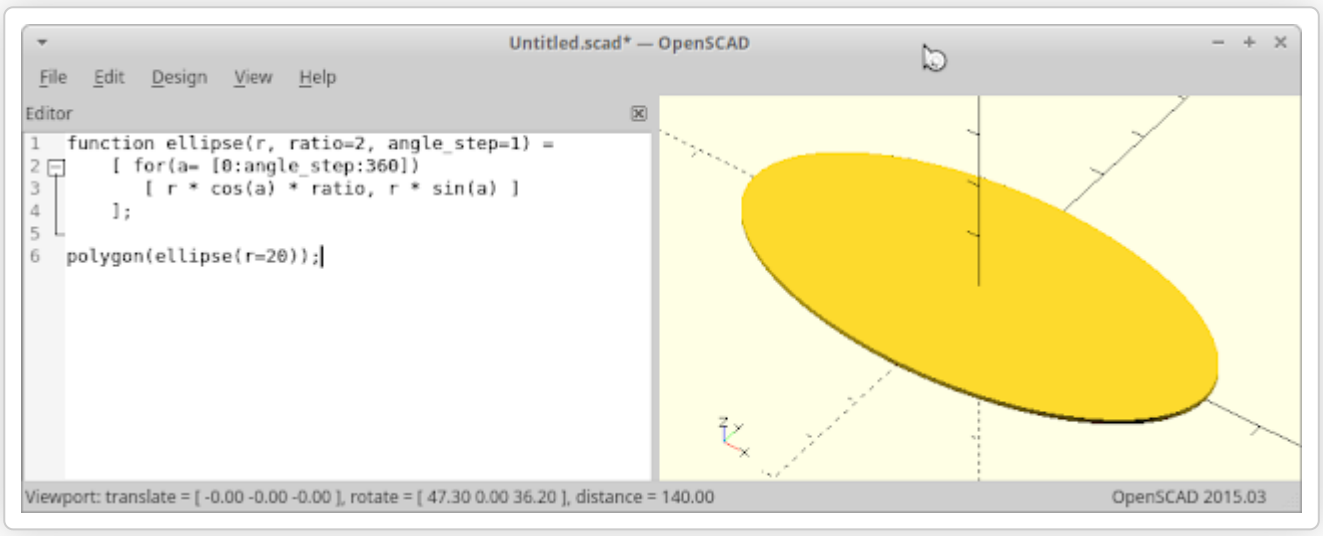
I wrote this introduction to Openscad to help people try and learn it in the first place, and from scratch. The order I presented the features may work as a step-by-step tutorial but they are probably very bad as a reference guide. Google may help bring back some parts, but once here I suggest that the best place to move forward is the official and comprehensive documentation. or just the short list of all the functions and features of the language. Update: see last paragraph for links.

As we have just seen, keep in mind that there are always many different ways to design one object.

But make sure you think about re-usability, parametric designs, and that you avoid pitfalls in the very first place. Because of lack of policies, there are just too many Openscad designs that are completely unusable... including some of my early ones, that I learnt the hard way. Parametric designs are a huge winner for 3D printers, because they let you tweak the tolerances easily with the reality of the first prototype. You need 0.2 mm more clearance? Openscad computes it all over again and you can print it again, it is amazingly efficient.

Yes, Openscad is a very powerful tool to design 3D shapes. It is not interactive, but it is nonetheless one of the very best, free and ubiquitous tools to design semi-industrial parts, like 3D printer links. It is not well suited to artists because it lacks fundamental primitives likes bezier curves or splines (well, almost since they can be coded separately), and no mouse will help you in the design process beyond rotating the part and zooming on it, and contrary to most of the heavy-loaded and often expensive CAD software.

But this is why Openscad is a powerful tool also in my opinion: I do not have to learn and keep practicing hundreds of shortcuts in Blender to be able to design whatever I need. I mostly never had something I could not make with openscad, and this is also why I keep on falling back to it, even when I wish it had a few more features like bevels, local variables or "professional" output formats and dimensions.



More features? Here is an example to make an ellipse() out of a function and thanks to polygon().
The deal is to generate the pairs of [x,y] points oursleves, which opens many possibilities.

Final note: so why article #4 out of #5? OK, OK, I originally intended to write one more article related to a some features I left out here, like functions and polygons (video here), command line invocation with pre-defined variables to automate the generation of parametric objects, animation (shaping with time such as in this script), basic routines and tricks that I often use when designing complex shapes (e.g. bezier curves), or scripts, including some that are useful to render the views (e.g. povray)...

But as you can see, more and more people provide high quality advanced information on Openscad nowadays, so you have multiple links to follow from now on. Nonetheless, I still think about this elusive fifth page, years after I wrote this serie ;)

Advanced libraries: "Round-Anything"

Primitives: https://github.com/nophead/NopSCADlib/blob/master/libtest.png

Posted by Jeremie Francois at 2:24 AM

Labels: review, software

No comments:

Post a Comment

To leave a comment, click the button below to sign in with Google.

SIGN IN WITH GOOGLE

Subscribe to: Post Comments (Atom)

About Me

Jeremie Francois

View my complete profile

Follow by RSS

Posts

Comments

Blog Archive

- ► 2017 (3)
- ► 2016 (7)
- ► 2015 (9)
- ▼ 2014 (18)
  - ▼ Nov 2014 (3)
    - How to use Openscad (4): children and advanced topics
    - How to use Openscad (3): iterations, extrusions an...
    - Fortune 500 teller? 3D printing trends, reports, a...
  - ► Sep 2014 (2)
  - ► Aug 2014 (2)
  - ► Jul 2014 (1)
  - ► Jun 2014 (2)
  - ► May 2014 (2)
  - ► Mar 2014 (2)
  - ► Jan 2014 (4)
- ► 2013 (14)
- ► 2012 (29)

More links? my curriculum, designed 3D objects, panoramas, pictures, company, and emaiil

This work is licensed CC BY-SA 4.0. Awesome Inc. theme. Powered by Blogger.